

Compte-rendu du projet Recherche Opérationnelle

Algorithmes de flot maximum et flot de coût minimum

Auteur : Mounia AREZZOUG

Établissement : Université Côte d'Azur

Année universitaire 2025–2026



Réseaux de flot

Ford-Fulkerson Bellman-Ford Dijkstra

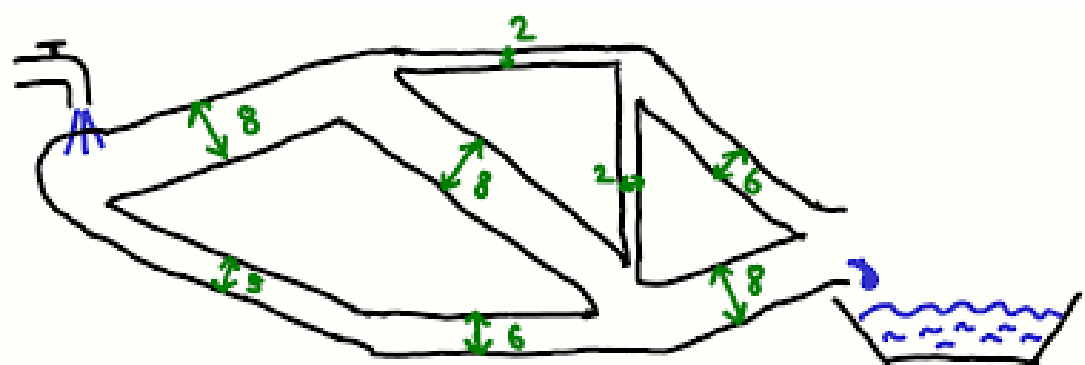


Table des matières

Introduction	3
1 Architecture et structure du projet	4
1.1 Structure générale du projet	4
1.2 Choix du langage Java	5
1.3 Représentation du graphe	5
1.4 Structure du graphe résiduel	6
1.5 Schéma UML du projet	7
2 Algorithme de flot maximum — Ford-Fulkerson	8
2.1 Principe général	8
2.2 Recherche de chemin augmentant	8
2.3 Mise à jour du graphe résiduel	9
2.4 Coupe minimum	9
2.5 Difficultés rencontrées	9
2.6 Tests et validation	9
3 Algorithmes de flot de coût minimum	12
3.1 Principe des chemins augmentants répétés	12
3.2 Version avec Bellman-Ford	12
3.3 Détection de cycles négatifs	13
3.4 Version avec Dijkstra et renormalisation des coûts	13
3.5 Difficultés rencontrées	14
3.6 Tests et validation	14
4 Génération du graphe résiduel avec Graphviz	16
4.1 Format .gv	16
4.2 Export du graphe résiduel	16
5 Compilation et exécution	17
5.1 Compilation du projet	17
5.2 Exécution du programme	17
5.3 Exécution de Ford-Fulkerson	17

5.4	Exécution du Min Cost Flow avec Bellman-Ford	18
5.5	Exécution du Min Cost Flow avec Dijkstra	18
5.6	Génération du PDF avec Graphviz	18
	Conclusion	19

Introduction

L'objectif de ce projet est d'implémenter plusieurs algorithmes fondamentaux de théorie des graphes et de recherche opérationnelle liés aux problèmes de flot dans les réseaux.

Le projet repose sur la lecture d'un graphe orienté pondéré depuis un fichier texte, puis sur l'application de différents algorithmes permettant soit de maximiser le flot entre une source et un puits, soit de minimiser le coût total du flot transporté.

Les algorithmes implémentés sont :

- Ford-Fulkerson pour le calcul du flot maximum et de la coupe minimum.
- Min Cost Flow par chemins augmentants répétés avec Bellman-Ford.
- Min Cost Flow par chemins augmentants répétés avec Dijkstra et renormalisation des coûts.
- Détection de cycles négatifs dans le graphe résiduel.

Le projet inclut également la génération automatique du graphe résiduel final au format Graphviz (`.gv`), permettant de visualiser graphiquement les résultats des algorithmes.

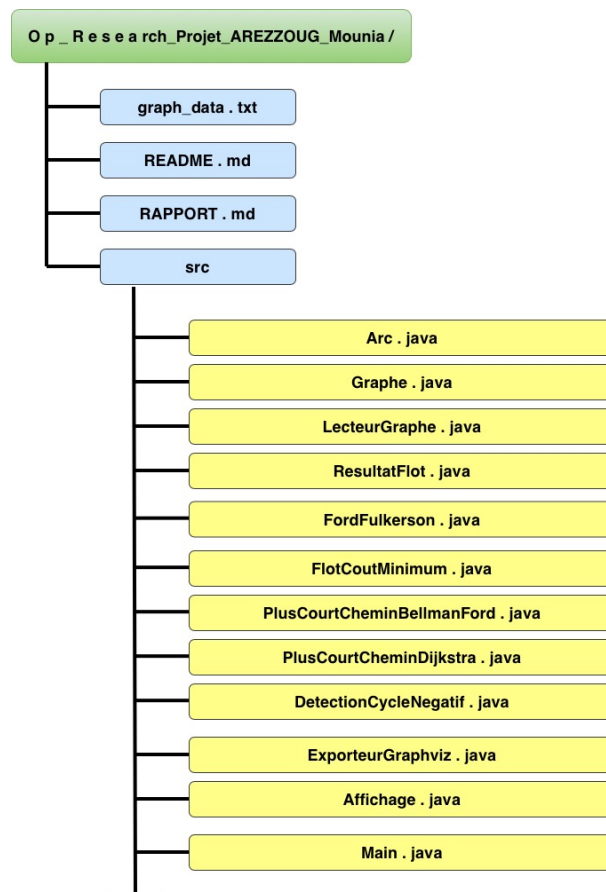
Chapitre 1

Architecture et structure du projet

1.1 Structure générale du projet

Le projet est organisé de manière modulaire afin de séparer clairement les différentes responsabilités : lecture du graphe, représentation des structures de données, algorithmes de flot, détection de cycles négatifs et export Graphviz.

Organisation du projet :



1.2 Choix du langage Java

Le projet a été entièrement développé en Java. Ce choix a été motivé par plusieurs raisons liées à la structure du projet et aux besoins algorithmiques des problèmes de flot.

Tout d'abord, Java offre une bonne gestion de la programmation orientée objet, ce qui facilite la modularisation du code. Les différentes composantes du projet (graphe, arcs, algorithmes de flot, export Graphviz et lecture des fichiers) ont pu être séparées en classes indépendantes, rendant le projet plus clair.

De plus, Java fournit des structures de données adaptées à la manipulation des graphes, comme les listes dynamiques (`ArrayList`), les files (`Queue`), les files de priorité (`PriorityQueue`) et les collections génériques, particulièrement utiles pour l'implémentation de BFS, Bellman-Ford et Dijkstra.

1.3 Représentation du graphe

Le graphe est représenté sous forme de liste d'adjacence. Chaque sommet possède une liste contenant tous les arcs sortants. Cette structure est plus efficace qu'une matrice d'adjacence pour les graphes creux, car elle évite de stocker des arcs inexistants.

La classe `Graphe` contient principalement :

```
private final List<List<Arc>> adjacence;
```

Chaque objet `Arc` représente un arc orienté du graphe.

Un arc contient :

- le sommet de départ ;
- le sommet d'arrivée ;
- la capacité maximale ;
- le coût unitaire ;
- le flot courant ;
- un pointeur vers son arc inverse.

Structure simplifiée d'un arc :

```
public class Arc {  
    public final int depart;  
    public final int arrivee;  
    public final int capacite;  
    public final int cout;  
    public int flot;  
    public Arc inverse;  
    public final boolean original;
```

1.4 Structure du graphe résiduel

Le graphe résiduel constitue l'élément central du projet.

Pour chaque arc original :

$$u \rightarrow v$$

un arc inverse est automatiquement ajouté :

$$v \rightarrow u$$

L'arc inverse permet d'annuler du flot lorsqu'un meilleur chemin est trouvé plus tard.

Exemple :

Arc original :

$$5 \rightarrow 0 \quad \text{capacité} = 10 \quad \text{coût} = 2$$

Arc inverse associé :

$$0 \rightarrow 5 \quad \text{capacité} = 0 \quad \text{coût} = -2$$

Si 4 unités de flot sont envoyées sur l'arc direct :

$$\text{flot}(5 \rightarrow 0) = 4$$

alors l'arc inverse devient :

$$\text{flot}(0 \rightarrow 5) = -4$$

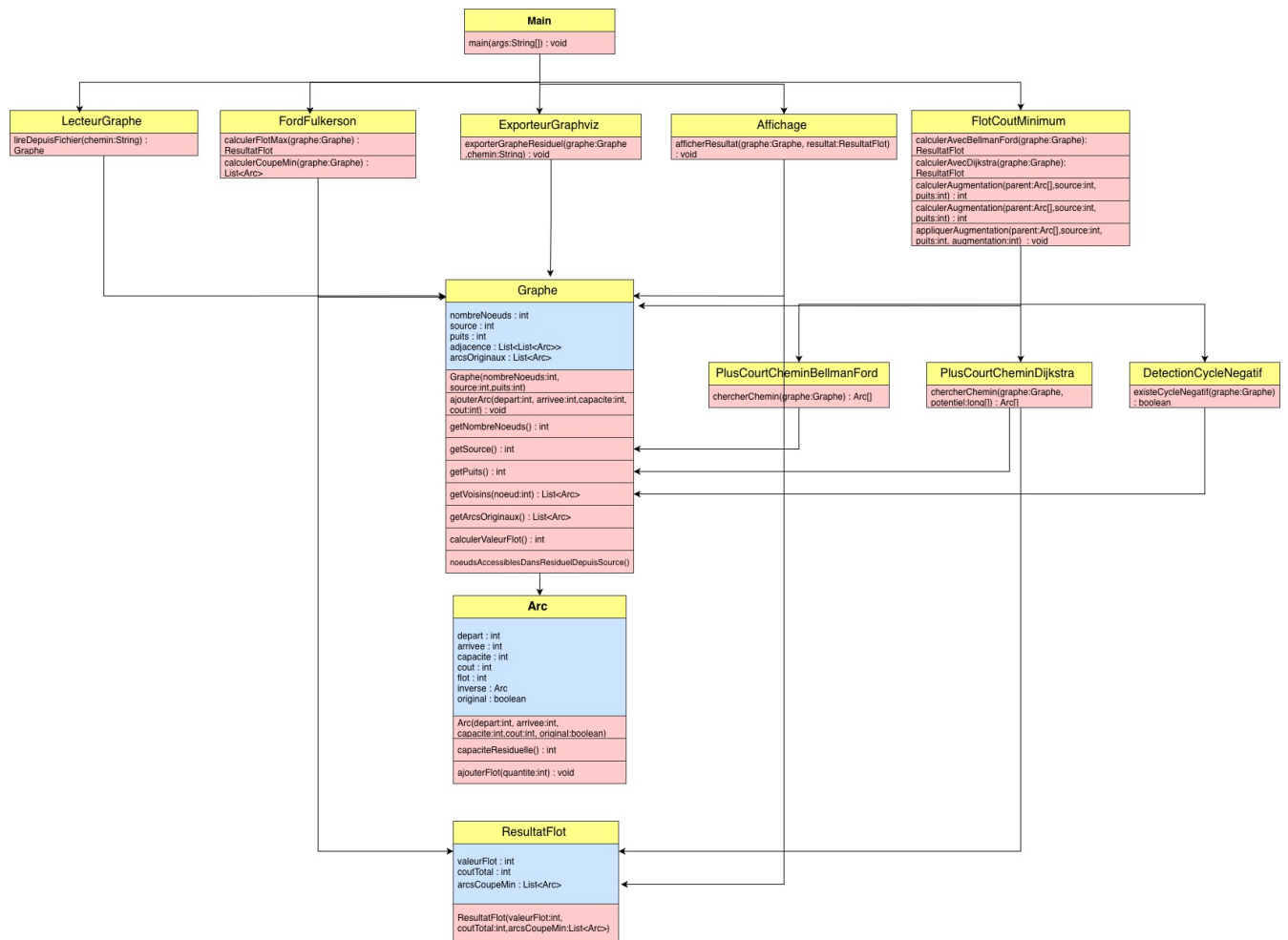
La capacité résiduelle d'un arc est calculée par :

$$\text{capacité résiduelle} = \text{capacité} - \text{flot}$$

Cette représentation permet :

- la recherche de chemins augmentants ;
- l'annulation partielle de flot ;
- le calcul des coûts résiduels ;
- la détection de cycles négatifs.

1.5 Schéma UML du projet



Chapitre 2

Algorithme de flot maximum — Ford-Fulkerson

2.1 Principe général

L'algorithme de Ford-Fulkerson permet de calculer le flot maximum entre une source s et un puits t .

Le principe consiste à rechercher des chemins augmentants dans le graphe résiduel.

Tant qu'un chemin augmentant existe :

- on détermine la capacité minimale du chemin ;
- on augmente le flot sur ce chemin ;
- on met à jour le graphe résiduel.

Lorsque plus aucun chemin augmentant n'existe, le flot obtenu est maximal.

2.2 Recherche de chemin augmentant

La recherche de chemin augmentant est effectuée avec un parcours en largeur (BFS).

Le BFS permet de trouver un chemin de la source vers le puits utilisant uniquement des arcs dont la capacité résiduelle est strictement positive.

Condition de parcours :

```
arc.capaciteResiduelle() > 0
```

Le tableau `parent` permet de reconstruire le chemin trouvé.

Exemple :

$$5 \rightarrow 0 \rightarrow 2 \rightarrow 4$$

Le chemin est ensuite parcouru à l'envers pour calculer la capacité minimale.

2.3 Mise à jour du graphe résiduel

Une fois le chemin augmentant trouvé, on calcule l'augmentation possible :

$$\text{augmentation} = \min(\text{capacités résiduelles du chemin})$$

Puis le flot est ajouté sur chaque arc du chemin.

Simultanément, l'arc inverse est mis à jour afin de permettre une éventuelle annulation future du flot.

Cette étape modifie directement le graphe résiduel.

2.4 Coupe minimum

Une fois le flot maximum obtenu, la coupe minimum est calculée à partir du graphe résiduel.

On effectue un parcours depuis la source en utilisant uniquement les arcs dont la capacité résiduelle est positive.

Les sommets accessibles constituent le côté source de la coupe.

Les arcs originaux reliant :

$$\text{sommet accessible} \rightarrow \text{sommet inaccessible}$$

forment alors la coupe minimum.

2.5 Difficultés rencontrées

Problème	Cause	Solution
Boucles infinies	Mauvaise mise à jour des arcs inverses	Synchronisation systématique des arcs directs et inverses
Flot incorrect	Chemins augmentants mal reconstruits	Utilisation du tableau parent
Mauvaise coupe minimum	Graphe résiduel mal parcouru	BFS final depuis la source

TABLE 2.1 – Difficultés rencontrées pour Ford-Fulkerson

2.6 Tests et validation

Plusieurs graphes de test ont été utilisés afin de vérifier :

— la validité du flot maximum ;

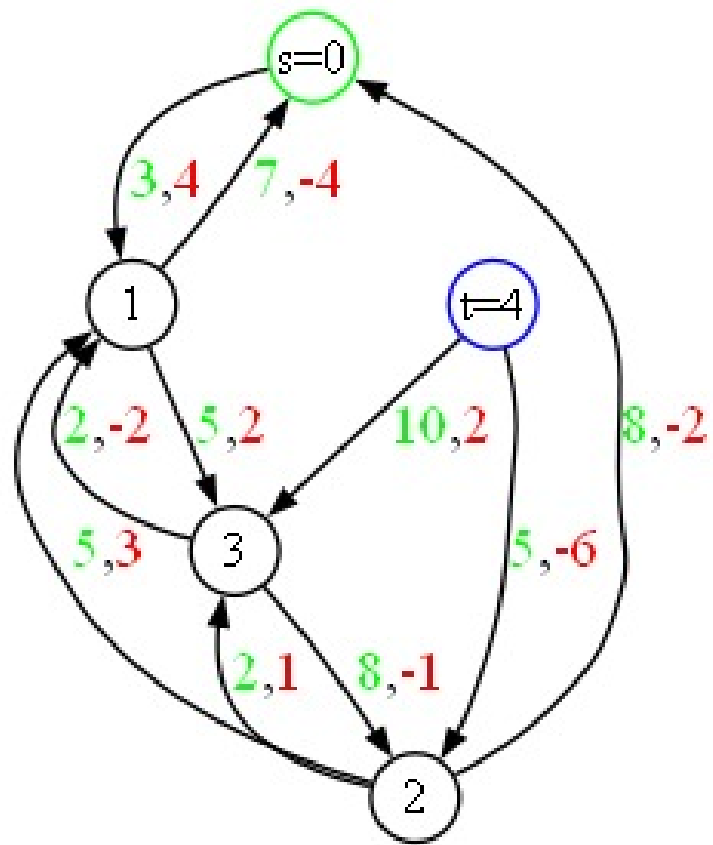
- la cohérence du graphe résiduel ;
- le calcul correct de la coupe minimum ;
- la mise à jour des arcs inverses.

Exemple de test :

```
6 10 5 4
5 0 10 2
5 1 8 4
0 1 5 5
0 2 5 2
1 0 4 1
1 3 10 4
2 1 7 1
2 3 6 2
2 4 3 1
3 4 14 3
```

Les résultats ont confirmé la cohérence du flot maximal obtenu et du graphe résiduel final :

```
mounia@MacBook-Pro-de-Mounia Op_Research_Projet_AREZZOUG_Mounia % java -cp out Main maxflow graph_
data.txt residuel_maxflow.gv
Valeur totale du flot = 15
Coût total du flot = 149
Flot sur chaque arc original :
 5 -> 0 : 7/10, coût=2
 5 -> 1 : 8/8, coût=4
 0 -> 1 : 2/5, coût=5
 0 -> 2 : 5/5, coût=2
 1 -> 0 : 0/4, coût=1
 1 -> 3 : 10/10, coût=4
 2 -> 1 : 0/7, coût=1
 2 -> 3 : 2/6, coût=2
 2 -> 4 : 3/3, coût=1
 3 -> 4 : 12/14, coût=3
Coupe minimum :
 0 -> 2
 1 -> 3
```



Chapitre 3

Algorithmes de flot de coût minimum

3.1 Principe des chemins augmentants répétés

Le problème du flot de coût minimum consiste à envoyer du flot de la source vers le puits tout en minimisant le coût total.

Le principe utilisé repose sur les chemins augmentants répétés.

À chaque itération :

- on cherche un chemin de coût minimum dans le graphe résiduel ;
- on augmente le flot sur ce chemin ;
- on met à jour les capacités résiduelles.

Le coût total du flot est calculé par :

$$\text{Coût total} = \sum (\text{flot} \times \text{coût unitaire})$$

3.2 Version avec Bellman-Ford

La première version utilise Bellman-Ford pour rechercher les chemins de coût minimum.

Bellman-Ford est particulièrement adapté car il accepte les arcs de coût négatif.

Dans le graphe résiduel, les arcs inverses possèdent un coût négatif :

$$\text{coût inverse} = -\text{coût original}$$

L'algorithme effectue plusieurs relaxations successives des arcs afin de calculer les plus courtes distances.

Condition de relaxation :

```
if (distance[v] > distance[u] + cout)
```

```
if (arc.capaciteResiduelle() > 0 && distance[arc.arrivee] > distance[u] + arc.cout)
```

3.3 Détection de cycles négatifs

Avant chaque recherche de chemin augmentant, une détection de cycle négatif est effectuée.

Un cycle négatif correspond à un cycle dont la somme des coûts est strictement négative.

Exemple :

$$1 \rightarrow 2 \quad \text{coût} = -5$$

$$2 \rightarrow 1 \quad \text{coût} = 1$$

Coût total du cycle :

$$-5 + 1 = -4$$

La présence d'un cycle négatif peut rendre le coût total arbitrairement petit.

La détection est réalisée grâce à une version adaptée de Bellman-Ford.

Après $n - 1$ relaxations, si une amélioration est encore possible, alors un cycle négatif existe.

3.4 Version avec Dijkstra et renormalisation des coûts

La seconde version du Min Cost Flow utilise Dijkstra.

Cependant, Dijkstra ne supporte pas les coûts négatifs.

Une technique de renormalisation des coûts par potentiels est donc utilisée.

Pour chaque arc :

$$\text{coût réduit} = \text{coût} + \text{potentiel}[u] - \text{potentiel}[v]$$

Les potentiels permettent de transformer les coûts afin de garantir des coûts réduits positifs ou nuls.

Le parcours de Dijkstra devient alors valide.

3.5 Difficultés rencontrées

Problème	Cause	Solution
Coûts négatifs incompatibles avec Dijkstra	Arcs inverses négatifs	Utilisation des potentiels
Mauvaise mise à jour des coûts réduits	Potentiels incorrects	Mise à jour après chaque itération
Détection incomplète des cycles	Relaxations insuffisantes	Vérification finale Bellman-Ford

TABLE 3.1 – Difficultés rencontrées pour le flot de coût minimum

3.6 Tests et validation

Des graphes avec coûts positifs et négatifs ont été utilisés pour tester :

- la validité des chemins minimums ;
- la cohérence du coût total ;
- la détection correcte des cycles négatifs.

2- Exemple de graphe contenant un cycle négatif :

```
4 5 0 3
0 1 10 2
1 2 10 -5
2 1 10 1
2 3 10 2
0 3 5 10
```

Cycle négatif :

$$1 \rightarrow 2 \rightarrow 1$$

Coût :

$$-5 + 1 = -4$$

Le programme détecte correctement ce cycle et interrompt l'exécution :

```
mounia@MacBook-Pro-de-Mounia Op_Rsearch_Projet_AREZZOUG_Mounia % java -cp out Main mcf-bf graph_CycleNegatif.txt residuel_mcf_bf.gv
Exception in thread "main" java.lang.IllegalStateException: Cycle négatif détecté dans le graphe r
ésiduel
    at FlotCoutMinimum.calculerAvecBellmanFord(FlotCoutMinimum.java:22)
    at Main.main(Main.java:22)
```

3- Exemple de graphe avec des couts négatifs (pour Bellman Ford) :

```
5 7 0 4
0 1 10 4
0 2 8 2
```

```

1 2 5 -3
1 3 7 2
2 3 10 1
2 4 5 6
3 4 10 -2

```

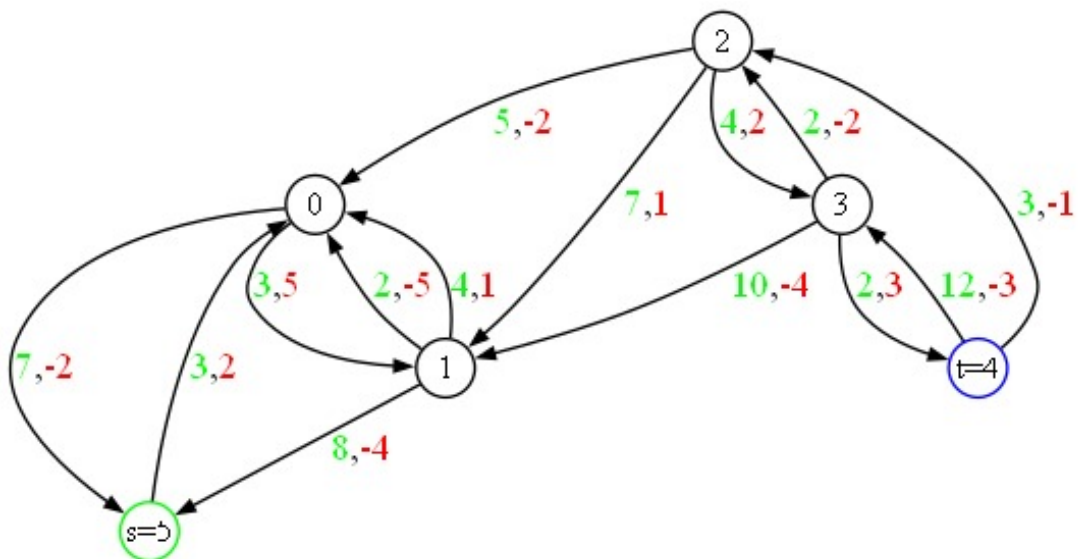
Le programme donne un résultat correct, commande lancée :

```
java -cp out Main mcf-bf graphe_negatif.txt residuel_mcf_bf.gv
```

```

mounia@MacBook-Pro-de-Mounia Op_Research_Projet_AREZZOUG_Mounia % java -cp out Main mcf-bf graphe_
negatif.txt residuel_mcf_bf.gv
Valeur totale du flot = 15
Coût total du flot = 51
Flot sur chaque arc original :
0 -> 1 : 7/10, coût=4
0 -> 2 : 8/8, coût=2
1 -> 2 : 5/5, coût=-3
1 -> 3 : 2/7, coût=2
2 -> 3 : 8/10, coût=1
2 -> 4 : 5/5, coût=6
3 -> 4 : 10/10, coût=-2

```



Chapitre 4

Génération du graphe résiduel avec Graphviz

4.1 Format .gv

Le graphe résiduel final est exporté au format Graphviz (.gv).

Ce format permet de générer automatiquement une représentation graphique du réseau.

Exemple :

```
digraph G {  
    5 -> 0 [label="10,2"]  
}
```

Chaque arc affiche :

- la capacité résiduelle ;
- le coût associé.

4.2 Export du graphe résiduel

La classe `ExporteurGraphviz` parcourt tous les arcs du graphe résiduel.

Seuls les arcs ayant une capacité résiduelle strictement positive sont exportés.

Les arcs sont ensuite écrits dans le fichier .gv.

La source et le puits sont affichés avec des couleurs différentes afin de faciliter la lecture.

Chapitre 5

Compilation et exécution

5.1 Compilation du projet

Depuis le dossier principal du projet, la compilation se fait avec la commande suivante :

```
javac -d out src/*.java
```

5.2 Exécution du programme

Le programme principal se lance ensuite avec la commande générale suivante :

```
java -cp out Main <algorithme> <fichier_entree> <fichier_sortie.  
gv>
```

où :

- <algorithme> correspond à l'algorithme choisi,
- <fichier_entree> est le fichier contenant le graphe,
- <fichier_sortie.gv> est le fichier Graphviz généré.

5.3 Exécution de Ford-Fulkerson

Pour calculer le flot maximum :

```
java -cp out Main maxflow graph_data.txt residuel_maxflow.gv
```

Le programme affiche :

- la valeur totale du flot maximum,
- les arcs de la coupe minimum,
- le flot sur chaque arc original.

5.4 Exécution du Min Cost Flow avec Bellman-Ford

Pour lancer la version Bellman-Ford :

```
java -cp out Main mcf-bf graphe_negatif.txt residuel_mcf_bf.gv
```

5.5 Exécution du Min Cost Flow avec Dijkstra

Pour lancer la version Dijkstra :

```
java -cp out Main mcf-dijkstra graph_data.txt  
    residuel_mcf_dijkstra.gv
```

5.6 Génération du PDF avec Graphviz

Une fois le fichier .gv généré, Graphviz permet de produire une représentation graphique du graphe résiduel :

```
dot residuel_mcf_bf.gv -Tpdf > res.pdf
```

Le fichier PDF obtenu contient une visualisation complète du graphe résiduel final.

Conclusion

Ce projet a permis d'implémenter plusieurs algorithmes fondamentaux de théorie des graphes et de recherche opérationnelle en Java.

L'utilisation d'un graphe résiduel avec arcs inverses a permis de gérer efficacement les problèmes de flot maximum et de flot de coût minimum.

Ford-Fulkerson permet de calculer le flot maximum et la coupe minimum, tandis que les versions du Min Cost Flow avec Bellman-Ford et Dijkstra permettent de minimiser le coût du transport dans le réseau.

La détection de cycles négatifs garantit la cohérence des coûts dans le graphe résiduel.

Enfin, l'export Graphviz facilite grandement la visualisation des résultats obtenus.

Ce projet illustre l'importance de la modularisation, des structures de données adaptées et des graphes résiduels dans les problèmes de flot en recherche opérationnelle.